

CASA and cult-cargo

Landman Bester

SARAO

2026-03-24

Stimela Recap — Cabs

- **Cabs** are atomic units of work — like functions
- Each cab has a well-defined **schema**: typed inputs and outputs
- A cab does one thing and does it well

Cab = function with a schema

Inputs → Cab → Outputs

Stimela Recap — Recipes

- **Recipes** also have a schema (inputs and outputs) — but they are more like scripts than functions
- A recipe strings multiple cabs together as a sequence of **steps**
- This sequence of steps defines a **pipeline**

Recipe = script that chains cabs into a pipeline

Step 1 (Cab A) → Step 2 (Cab B) → Step 3 (Cab C)

The `_include` Construct

`_include` lives at the top of a recipe and **merges** external YAML into it — bringing in cab definitions from CultCargo or your own files.

```yaml
_include:
  - (cultcargo)wsclean.yml    # curated cab from cult-cargo
  - my-cabs.yml               # user-defined cabs
```

The `(module)file.yml` syntax resolves `file.yml` from inside an installed Python package — so `(cultcargo)wsclean.yml` looks for `wsclean.yml` inside the `cultcargo` package. The file must exist in that package or the include fails.

> Think of it like Python's `dict.update` — keys from included files are merged in, and you can override any leaf value.

`_include` as a Merge

```
# _include merges like dict.update
base = {"cabs": {"wsclean": {"command": "wsclean"}}}
override = {"cabs": {"wsclean": {"command": "~/my/wsclean"}}}

base["cabs"]["wsclean"].update(override["cabs"]["wsclean"])
# => {"cabs": {"wsclean": {"command": "~/my/wsclean"}}}
```

Stimela equivalent:

```
_include:
  - (cultcargo)wsclean.yml

cabs:
  wsclean:
    command: ~/my/wsclean
```

Subsections **merge**, leaf values **overwrite** — just like nested `dict.update`.

Cab Flavours

Cabs come in four flavours, set by the `flavour` field:

Flavour	`command` is...	Example
`binary`	an executable	`/bin/cp`
`python`	a callable	`os.getloadavg`
`python-code`	inline code	`c = a + b`
`casa-task`	a CASA task	`flagdata`

```yaml
cabs:
  cp:
    flavour: binary
    command: /bin/cp
```

Binary — runs an executable

```yaml
cabs:
  add:
    flavour: python
    command: module.function
```

Python — calls a python function

CASA5 vs CASA6 Cabs

CASA5 — monolithic

- Shipped its own Python 2.7 + IPython shell
- Tasks only available inside the CASA interpreter
- Stimela flavour: `casa-task` — launches an external CASA binary to run the task

CASA6 — modular

- `pip install casatools casatasks` into any Python 3 environment
- Tasks are regular Python callables
- Stimela flavour: `python` — just imports and calls the function directly
- Proper exception handling

`cult-cargo` only ships CASA6-style (`python` flavour) cabs. You would need to write your own if you wanted to use CASA5.

Discovering CASA Cabs in cult-cargo

List all available CASA cabs (note `-l` flag):

```
stimela doc -l cultcargo.casa::
```

Get help on a specific cab:

```
stimela doc cultcargo.casa:: casa.bandpass
```

`cultcargo.casa::` scopes the query to the `casa` module in the `cultcargo` package. Get help on a specific cab by dropping `-l` and adding appending the cab name (in this case `casa.bandpass`).

Example - Getting the Data

Download the data available here to get started.

Extract the data using

```
mkdir data
tar -xvf ARIWS.INTROFIMAGING.TriA.1559937657.tar.gz -C data/
ls -lrt data/
1559937657.PRIMARY.1.0.1.1.ghz.avg832khz32s.12ants.scan2.3.ms  # pr
1559937657.SECONDARY.1.0.1.1.ghz.avg832khz32s.12ants.ms  # secondar
1559937657.RXCJ1638.2-6420.1.0.1.1.ghz.avg832khz12s.12ants.scan2.ms
```

Example - Building the Recipe

1. `listobs` — list the details of all observations
2. `flagdata.save` — backup the FLAG column in each MS
3. `flagsummary` — report how much data are flagged in each MS
4. `plotms` — inspect off-diagonal correlations: amplitude vs frequency for baselines 1–100 m
5. `flagdata` — flag corrupted data
6. **KGB calibration** — solve for gains on the primary calibrator field
7. **Solution transfer** — transfer solutions to secondary
8. **Phase calibration** — solve for the phases on the secondary calibrator field
9. **Quality assurance** — make QA plots to ensure the calibration was successful
10. **Image + selfcal** — image and self-calibrate the target field

Thanks!

lbester@sarao.ac.za